

Requirements Document

Project Title

Car Rental Database and Management System

Project Purpose

The purpose of this project is to design and build a relational database system for a car rental business. The system will store and manage information about customers, staff members, vehicles, rentals, payments, and maintenance records.

This project is being used for both CS4092 Database Design and Development and CS3003 Programming Languages. The database portion focuses on the design and implementation of the relational database. The programming languages portion focuses on building a Python command-line application that interacts with the database.

System Overview

The Car Rental Database and Management System allows staff members to manage the basic operations of a car rental company. Staff members can add customers, add vehicles, view available vehicles, search for vehicles, rent vehicles to customers, return vehicles, view rental history, record payments, and add maintenance records.

The system uses a relational database to organize and connect all important information. A Python command-line interface will be used to interact with the database.

Users

The system has one main user type:

Staff Member

A staff member uses the system to manage customers, vehicles, rentals, payments, and maintenance records.

Staff members can:

- Add new customers
- Add new vehicles
- View available vehicles
- Search vehicles
- Create rental records
- Return vehicles
- View rental history
- View payments

- Add maintenance records

Data Requirements

The system must store information about customers, staff members, vehicles, rentals, payments, and maintenance.

Customer Data

The system must store:

- Customer ID
- First name
- Last name
- Phone number
- Email address
- Driver's license number

Staff Data

The system must store:

- Staff ID
- First name
- Last name
- Phone number
- Email address
- Role

Vehicle Data

The system must store:

- Vehicle ID
- Make
- Model
- Year
- License plate number
- Daily rental rate
- Vehicle status

Vehicle status can include:

- Available
- Rented
- Maintenance

Rental Data

The system must store:

- Rental ID
- Customer ID
- Vehicle ID
- Staff ID
- Start date
- End date
- Return date
- Rental status

Rental status can include:

- Active
- Returned

Payment Data

The system must store:

- Payment ID
- Rental ID
- Payment amount
- Payment date
- Payment method

Payment methods can include:

- Cash
- Credit Card
- Debit Card

Maintenance Data

The system must store:

- Maintenance ID
- Vehicle ID
- Maintenance date
- Maintenance description
- Maintenance cost
- Maintenance status

Maintenance status can include:

- Scheduled
- Completed

Use Cases

Use Case 1: View Available Vehicles

A staff member can view a list of vehicles that are currently available for rent.

Use Case 2: Search Vehicles

A staff member can search for vehicles by make or model.

Use Case 3: Add Customer

A staff member can add a new customer to the system by entering the customer's name, phone number, email address, and driver's license number.

Use Case 4: Add Vehicle

A staff member can add a new vehicle to the system by entering the vehicle make, model, year, license plate number, and daily rental rate.

Use Case 5: Rent Vehicle

A staff member can create a rental record when a customer rents a vehicle. The system will connect the customer, vehicle, and staff member to the rental record.

Use Case 6: Return Vehicle

A staff member can update a rental record when a customer returns a vehicle. The system will mark the rental as returned and mark the vehicle as available again.

Use Case 7: View Rental History

A staff member can view rental history, including the customer name, vehicle information, rental dates, return date, and rental status.

Use Case 8: View Payments

A staff member can view payment information connected to rental records.

Use Case 9: Add Maintenance Record

A staff member can add a maintenance record for a vehicle. The system will store the maintenance date, description, cost, and status.

Business Rules

1. Each customer can have many rentals.
2. Each rental belongs to one customer.

3. Each rental is for one vehicle.
4. Each vehicle can appear in many rental records over time.
5. A vehicle can only have one active rental at a time.
6. A vehicle must be marked as rented when it is rented by a customer.
7. A vehicle must be marked as available when it is returned.
8. Each rental is managed by one staff member.
9. Each staff member can manage many rentals.
10. Each rental can have one or more payments.
11. Each payment belongs to one rental.
12. Each vehicle can have many maintenance records.
13. A maintenance record belongs to one vehicle.
14. A vehicle can be marked as maintenance when it is being repaired or inspected.

Functional Requirements

The system should allow staff members to:

1. Initialize the database.
2. View all available vehicles.
3. Search vehicles by make or model.
4. Add a new customer.
5. Add a new vehicle.
6. Create a rental record.
7. Return a rented vehicle.
8. View rental history.
9. View payment records.
10. Add maintenance records.

Non-Functional Requirements

The system should be:

- Easy to use through a command-line menu.
- Organized using separate files for database logic, program logic, and model classes.
- Reliable enough to store and retrieve rental data correctly.
- Designed with clear primary keys and foreign keys.
- Simple enough to run locally using Python and SQLite.

Assumptions

This system is designed as a small educational project, not a full commercial car rental system.

The system assumes that staff members are the main users of the program.

The system does not include online payment processing.

The system does not include customer login accounts.

The system does not include a web front end.

Expected Output

At the end of the project, the system will include:

- A requirements document
- An ER diagram
- A relational schema design
- SQL table creation statements
- SQL insert statements with sample data
- At least three SQL queries
- A Python command-line program
- A GitHub repository
- A video demonstration